An Elementary Method for Fast Modular Exponentiation With Factored Modulus

Anay Aggarwal[1] Manu Isaacs[1]

[1]Euler Circle, Mountain View, CA 94040

West Coast Number Theory, December 2023

The Main Result

Theorem 1

Let m be a positive integer with known factorization $p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ and let a, n be positive integers such that $\gcd(a, m) = 1$. For each $1 \leq i \leq k$, choose an integer parameter $t_i \in [0, e_i]$. We may then compute $a^n \pmod{m}$ in

$$O\left(\max\left(\frac{e_i}{t_i}\right) + \sum_{i=1}^{k} t_i \log p_i\right)$$

steps.

The starting point is to write $T = \prod_{1 \leq i \leq k} p_i^{t_i}$, and use the division algorithm to decompose n modulo $\phi(T)$. There are many clever tricks that we can use after making this decomposition.

Analysis

In most cases, taking $t_i = 1$ for all i, i.e. $T = \mathrm{rad}(m)$, is particularly efficient. The asymptotic complexity of this choice is

$$O(\log(\mathrm{rad}(m)) + \max(e_i)).$$

The goal is to compare this to $O(\log m)$. Asymptotically, we expect $\max(e_i)$ to be $O(1)$. In particular, we expect it to be $c = 1 + \sum_{k=2}^{\infty} \left(1 - \frac{1}{\zeta(k)}\right) \approx 1.7052\cdots$ (Niven 1969).

Analysis

To determine precise asymptotics, it is difficult to work with the ratio $\frac{\log m}{\log(\mathrm{rad}(m))}$. Instead work with $f(m) = \frac{m}{\mathrm{rad}(m)}$, which is multiplicative. Note $f(p^e) = p^{e-1}$, so we may compute its Dirichlet series as

$$F(s) = \sum_{n=1}^{\infty} \frac{f(n)}{n^s} = \prod_p \left(1 + \sum_{k=1}^{\infty} \frac{p^{k-1}}{p^{ks}}\right) = \prod_p \left(1 + \frac{1}{p^s - p}\right),$$

by way of the Euler Product. Thus

$$\frac{F(s)}{\zeta(s)} = \prod_p \left(1 + \frac{p-1}{p^{2s} - p^{s+1}}\right).$$

Sending $s \to 1^+$ on the real axis, the product tends to infinity. This is enough to deduce that $\sum_{m \leq x} f(m)$ is not $O(x)$.

Analysis

In 2013, Robert and Tenebaum proved the stronger fact that for large x,

$$\sum_{n \leq x} f(n) = x \exp\left((1 + o(1))\sqrt{\frac{8 \log x}{\log \log x}}\right),$$

Which isn't $O(x(\log(x))^A)$ for any A.

Analysis

There are specific families of m for which the algorithm performs particularly well. A basic corollary of theorem 1 is the following:

Corollary

Let m be a positive integer with known factorization $p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ such that all primes p_i have the same bit length as an integer P, and all e_i are $\Omega(\sqrt{\log P})$. Let a, n be positive integers such that $\gcd(a, m) = 1$. We may then compute $a^n \pmod m$ in $O(\sqrt{\log m})$ steps.

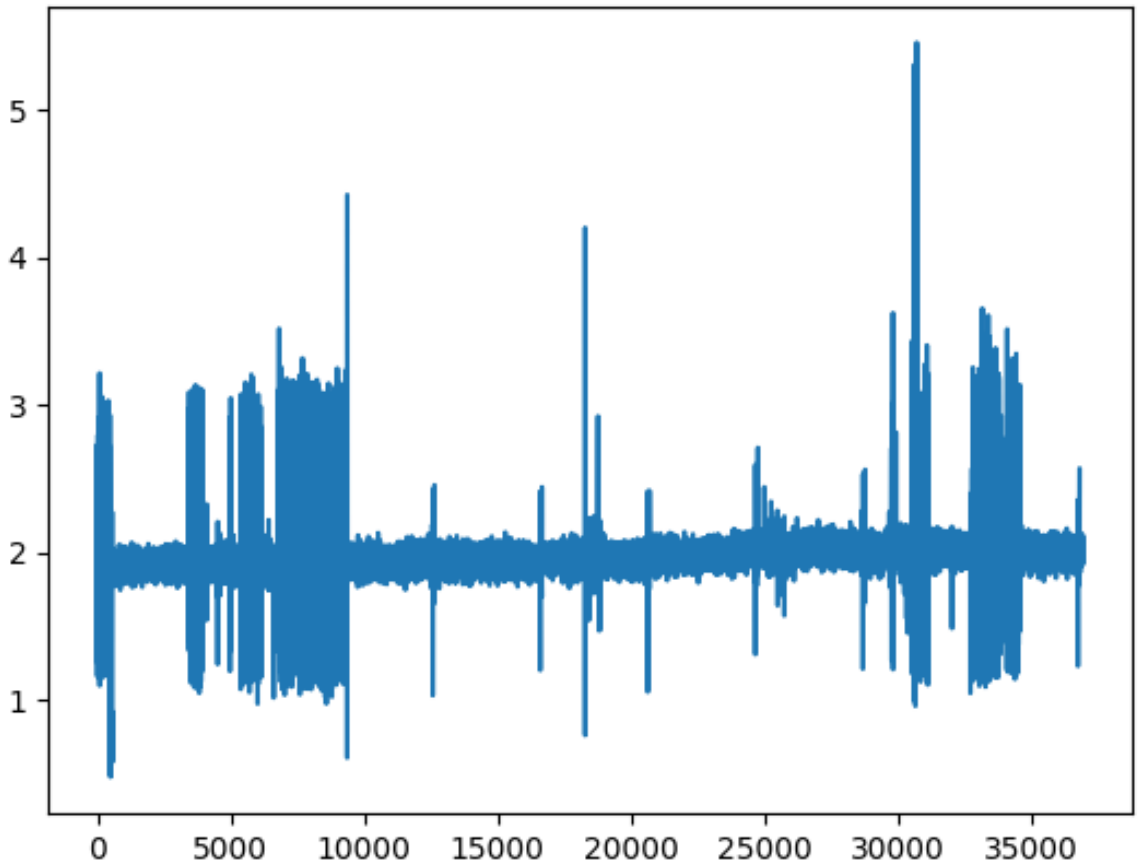
A simple example of such an m is prime powers! Take $m = p^k$ for some prime p and some $k = \Omega(\sqrt{\log P})$. Then our algorithm takes $O(\sqrt{k \log p})$ steps (with a suitable choice of t_i) as opposed to the standard $O(k \log p)$.

Programmatical Testing

- Testing against Python's built-in pow function
- Choose $m = p^k$
- Choose $k \in [\sqrt{\log p} - \sqrt{\log \log p}, \sqrt{\log p} + \sqrt{\log \log p}]$ randomly.
- Choose $a \in [p^k/2, p^k], \gcd(a, p) = 1$ randomly.
- Choose $n = O(p^k)$ randomly.

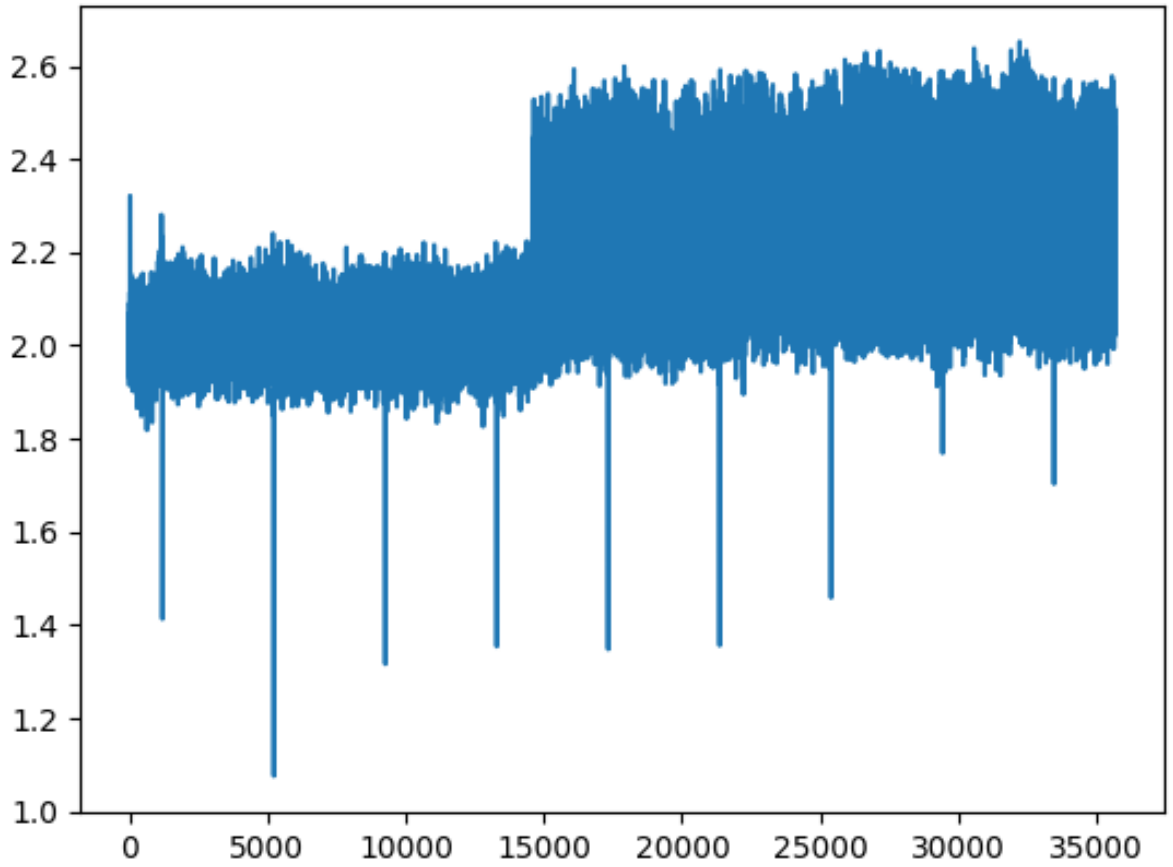
Analysis

The below graph has the ratio on the y-axis, and $p_{n+0.7\cdot 10^5}$ on the x-axis (for $0 \leq n \leq 3.5 \cdot 10^4$). For small primes, there's a lot of variance in the ratio that we win by.



Analysis

The below graph has the ratio on the y-axis, and $p_{n+3.5 \cdot 10^5}$ on the x-axis (for $0 \leq n \leq 3.5 \cdot 10^4$). For larger primes, there seems to be less variance.



Cryptographic Applications?

- Diffie-Hellmann? Schnorr's signature algorithm?
- Composite DLP
- Let's analyze the case where $m = p^k$, general analysis is difficult.
- Compare the EH ratio $\frac{\#\text{Encryption Steps}}{\#\text{Hacking Steps}}$.
- $\mathbb{Z}^*_{p^k}$ (where $k = \Omega(\sqrt{\log p})$) versus $\mathbb{Z}^*_q$.

The ratio for $\mathbb{Z}_q^*$

This ratio is well-known for $\mathbb{Z}_q^*$. Using the General Number Field Sieve, the ratio is asymptotically

$$\frac{\log q}{L_q[1/3, (64/9)^{1/3}]},$$

where we use the *L-notation*

$$L_n[\alpha, c] = e^{(c+o(1))(\log n)^{\alpha}(\log\log a)^{1-\alpha}}.$$

The ratio for $\mathbb{Z}^*_{p^k}$

With Pohlig-Hellmann as the hacking algorithm, we have the following theorem

Theorem

Suppose $p-1$ factors canonically as $\prod r_i^{s_i}$, then the ratio is asymptotically

$$\frac{\sqrt{k \log p}}{k^2 \log p + k\sqrt{p} + k \log p \sum s_i + \sum s_i \sqrt{r_i}}.$$

Using some estimates from Niven's paper and Hardy-Ramanujan, we may roughly compute what we "expect" this to be.

The ratio for $\mathbb{Z}^*_{p^k}$

We will need to introduce the constant $C = \frac{1}{2c}$ (where c is Niven's constant), and the modified L-notation

$$L^*_n[\alpha, c] = e^{(c+o(1))(\log n)^\alpha (\log\log n)^{-\alpha}} = L_n[\alpha, c]^{1/\log\log n}.$$

With this introduction, we "expect" this ratio to be

$$\frac{\sqrt{k \log p}}{k^2 \log p + k\sqrt{p} + k \log p \log\log p + L^*_p[1, C] \log\log p}.$$

Summary

- When the encryption time is the same, i.e. $\log q \approx \sqrt{k \log p}$, we win.
- In practice, we should be comparing $q \approx p^k$ (due to multiplication complexity). We lose here.